

Influencia del factor de redimensionamiento y el umbral de carga en tablas hash dinámicas con hashing universal

José Cisternas Gabriel Olarte Bernardo del Aguila Wara Murillo

Algorítmica II — 11 de septiembre de 2026

Resumen

Estudiamos experimentalmente cómo el factor multiplicativo de crecimiento γ y el umbral de carga τ afectan una tabla hash dinámica con encadenamiento separado y hashing universal. La investigación distingue tres conceptos que suelen confundirse: costo amortizado sobre una secuencia, esperanza respecto de la elección aleatoria de la función hash y promedio empírico entre semillas. Implementamos desde cero una tabla instrumentada, ejecutamos un diseño factorial bloqueado y registramos migraciones, colisiones por pares, comparaciones, memoria estable y pico, y tiempo. Contrastamos las mediciones con las predicciones $1/(\gamma - 1)$ para migraciones, γ/τ para reserva de buckets y $n(n - 1)/(2m)$ como cota esperada de pares bajo universalidad. El estudio busca una frontera de compromiso, no declarar una configuración universalmente óptima. Un experimento secundario de sondeo lineal muestra por qué las nociones de colisión no pueden trasladarse entre esquemas sin cautela.

Palabras clave: hashing universal, análisis amortizado, tablas hash dinámicas, colisiones, reproducibilidad.

1. Introducción

Las tablas hash ofrecen operaciones rápidas cuando la función dispersa adecuadamente las claves, pero una implementación dinámica debe decidir cuándo crecer y cuánto espacio reservar. Un crecimiento pequeño evita capacidad ociosa, aunque repite migraciones; uno grande reduce reconstrucciones, pero puede elevar la memoria pico. De forma análoga, un umbral bajo mantiene cadenas cortas a costa de más buckets. Estas decisiones relacionan de manera directa dos unidades del bloque final de Algorítmica II: análisis amortizado y algoritmos aleatorizados.

La literatura clásica proporciona las piezas por separado. Carter y Wegman formalizan familias universales [5]; Tarjan organiza el análisis amortizado de secuencias [14]; y Larson estudia tablas dinámicas y expansión [8]. Evaluaciones posteriores muestran que rendimiento, carga, arquitectura y representación interactúan [12, 3]. Nuestro aporte es integrar esas piezas en un factorial reproducible sobre la misma implementación.

1.1. Pregunta e hipótesis

Pregunta. ¿Cómo influyen el factor de redimensionamiento y el umbral de carga en el costo amortizado, el consumo de memoria y el número esperado de colisiones de una tabla hash dinámica con hashing universal?

H1. Al aumentar γ deben disminuir el número de resizes y las migraciones amortizadas aproximadamente como $1/(\gamma - 1)$, pero aumentar la capacidad ociosa y el pico. Al disminuir τ deben caer la carga y las colisiones, aumentando la reserva. La interacción formará una frontera de Pareto.

H0. Una vez controlada la carga realizada α , γ y τ no tienen efecto directo adicional sobre los pares que colisionan; los efectos residuales provienen de la trayectoria de cargas, redondeo, renovación del hash o variabilidad experimental.

2. Marco teórico

2.1. Hashing universal y colisiones

Usamos la familia modular

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m,$$

donde $p = 2^{61} - 1$, $a \in \{1, \dots, p-1\}$ y $b \in \{0, \dots, p-1\}$. Los parámetros se eligen con una semilla registrada y se renuevan en cada rehash. Para un conjunto fijo, una familia universal limita la probabilidad de colisión de cada par por $1/m$ [5]. Por linealidad de la esperanza, si

$$C_{pares} = \sum_{j=0}^{m-1} \binom{L_j}{2},$$

entonces $\mathbb{E}[C_{pares}] \leq n(n-1)/(2m)$. El contador se actualiza en $O(1)$: insertar en una cadena de longitud L agrega L pares.

Esta garantía no implica que una ejecución particular quede debajo de la esperanza ni permite atribuir sin más las fórmulas de sondeo lineal a una familia con independencia limitada. Resultados sobre tabulación simple y sondeo lineal muestran que el grado y estructura de independencia importan [11, 7].

2.2. Costo amortizado

Si una tabla de capacidad m_j crece a aproximadamente γm_j cuando contiene τm_j elementos, el ciclo siguiente admite cerca de $\tau m_j(\gamma-1)$ inserciones y la reconstrucción mueve cerca de τm_j entradas. Por ello, el término dominante de migración por inserción es

$$\frac{\tau m_j}{\tau m_j(\gamma-1)} = \frac{1}{\gamma-1}.$$

Esta es una garantía sobre la secuencia, independiente de una distribución de entradas. En contraste, las colisiones esperadas dependen de la elección aleatoria de h . El promedio medido sobre 30 semillas es un estimador empírico y no sustituye ninguna de las dos demostraciones.

Además del método agregado anterior, la interpretación contable puede cobrar a cada inserción créditos suficientes para financiar futuras migraciones. Una función potencial basada sólo en ocupación requiere cuidado porque la política combina γ , τ y redondeo a primos; por eso la usamos como perspectiva complementaria y no como prueba principal.

2.3. Memoria y carga

Inmediatamente después de crecer, $\alpha_{post} \approx \tau/\gamma$. Los buckets por elemento se aproximan a γ/τ y la fracción ociosa a $1 - \tau/\gamma$. Mientras coexisten la tabla vieja y la nueva, el pico estructural se aproxima a $(1+\gamma)/\tau$ buckets por elemento. Estas identidades motivan medir estado estable y pico por separado. Trabajos sobre arreglos redimensionables estudian precisamente la frontera tiempo-espacio [4, 13].

3. Trabajos relacionados

Larson conecta carga, expansión y costo de reorganización en hashing dinámico [8]. Dietzfelbinger et al. demuestran garantías fuertes para hashing perfecto dinámico, aunque con una estructura distinta [6]. Pagh y Rodler relacionan rehash aleatorio con cuckoo hashing [10]. Investigaciones recientes exploran estructuras dinámicas en GPU, alta carga y optimización simultánea de criterios [9, 2, 1]. No trasladamos directamente sus resultados: arquitectura, borrados y resolución de colisiones cambian la interpretación de τ .

4. Metodología

4.1. Implementación y tratamientos

La tabla principal usa encadenamiento separado, claves enteras y capacidad inicial prima $m_0 = 11$. Antes de insertar, si $(n + 1)/m > \tau$, se asigna

$$m' = \text{next_prime}(\lceil \gamma m \rceil),$$

se elige un nuevo (a, b) y se migran las claves. El factorial contiene $\gamma \in \{1.25, 1.5, 2, 3, 4\}$, $\tau \in \{0.50, 0.70, 0.80, 0.90\}$ y $N \in \{10^3, 10^4, 10^5\}$. Cada celda utiliza 30 semillas; son 1800 ejecuciones para la familia aleatoria principal.

La unidad experimental es ‘(gamma,tau,N,key_seed,order_seed,hash_seed)’. Dentro de un bloque, todos los tratamientos reciben exactamente el mismo conjunto y orden de claves. El orden de los 20 tratamientos se aleatoriza para repartir deriva térmica y carga externa. Un piloto previo usa cinco semillas y $N \leq 10^4$.

4.2. Variables e instrumentación

Las dependientes primarias son elementos migrados, resizes, evaluaciones hash, comparaciones, eventos de bucket ocupado, pares finales, longitud máxima de cadena, slots estables/pico y bytes estructurales. Se registran además pico de `tracemalloc`, nanosegundos totales y latencias $p50$, $p95$, $p99$ y máxima. Se usa `perf_counter_ns`; el calentamiento ocurre fuera de la región medida y el estado del recolector queda en el CSV.

No combinamos los contadores con pesos arbitrarios. Cada componente se informa por separado. `sys.getsizeof` recorre objetos sin duplicar identidades; `tracemalloc` es complementario y no equivale a RSS. El commit, Python, plataforma y CPU quedan junto a cada observación.

4.3. Validación y análisis

Las pruebas verifican tamaño, unicidad, ubicación, carga, conservación, capacidad prima, renovación de hash y equivalencia del contador incremental con $\sum_j \binom{L_j}{2}$. Para cada celda reportamos media, mediana, desviación e intervalo bootstrap del 95 %. La interpretación prioriza tamaño de efecto y orden de tendencias. Ocho figuras cubren heatmaps, trayectoria, cota universal, frecuencia teórica, Pareto y variabilidad entre semillas.

5. Resultados

La corrida definitiva contiene **1800** unidades experimentales y 30 repeticiones por celda. Para $N = 100000$, la correlación de rangos entre γ y migraciones por inserción, promediando τ , fue -1.000; entre τ y colisiones por pares por clave, promediando γ , fue 0.949. El cociente medio entre

pares observados y la cota universal fue 0.964. Estos valores describen la muestra de semillas y no convierten una esperanza en cota determinista.

La menor migración media apareció en $(\gamma, \tau) = (4, 0.70)$; el menor uso estructural final en $(1.5, 0.90)$; y la menor tasa de pares en $(4, 0.50)$. Que estos puntos no sean necesariamente el mismo tratamiento aporta evidencia empírica del compromiso multiobjetivo.

Cuadro 1: Promedios por γ para $N = 100000$, agregados sobre τ y semillas.

γ	migr./ins.	bytes/elem.	pares/clave	resizes	ns/oper.
1.25	4.508	157.7	0.333	40.2	66080.8
1.5	2.413	166.1	0.289	23.0	43657.2
2	1.491	187.3	0.222	14.0	33174.9
3	0.929	218.5	0.204	9.0	29056.5
4	0.723	276.8	0.189	7.2	27944.1

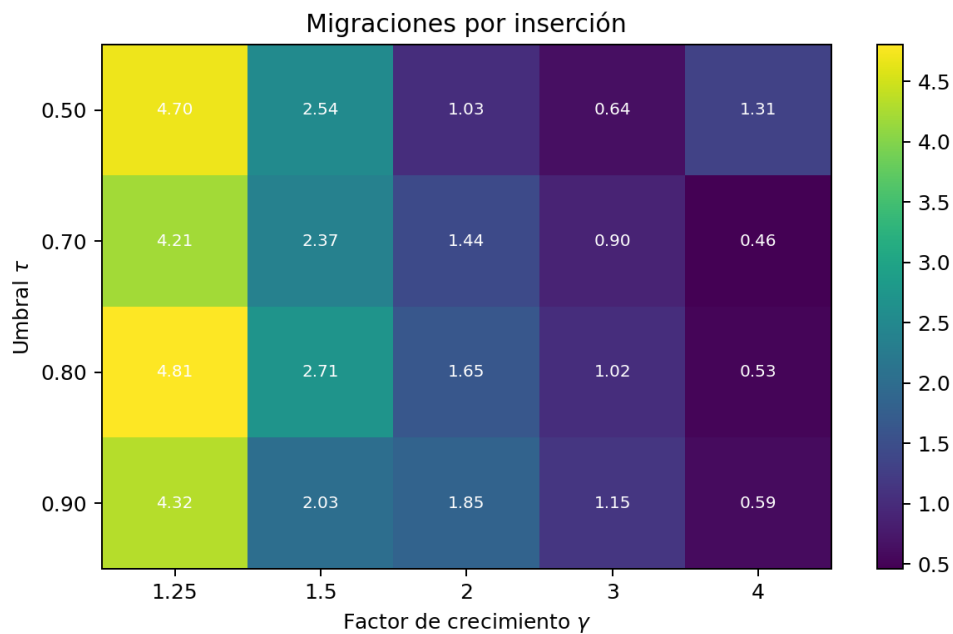


Figura 1: Migraciones normalizadas para el mayor tamaño.

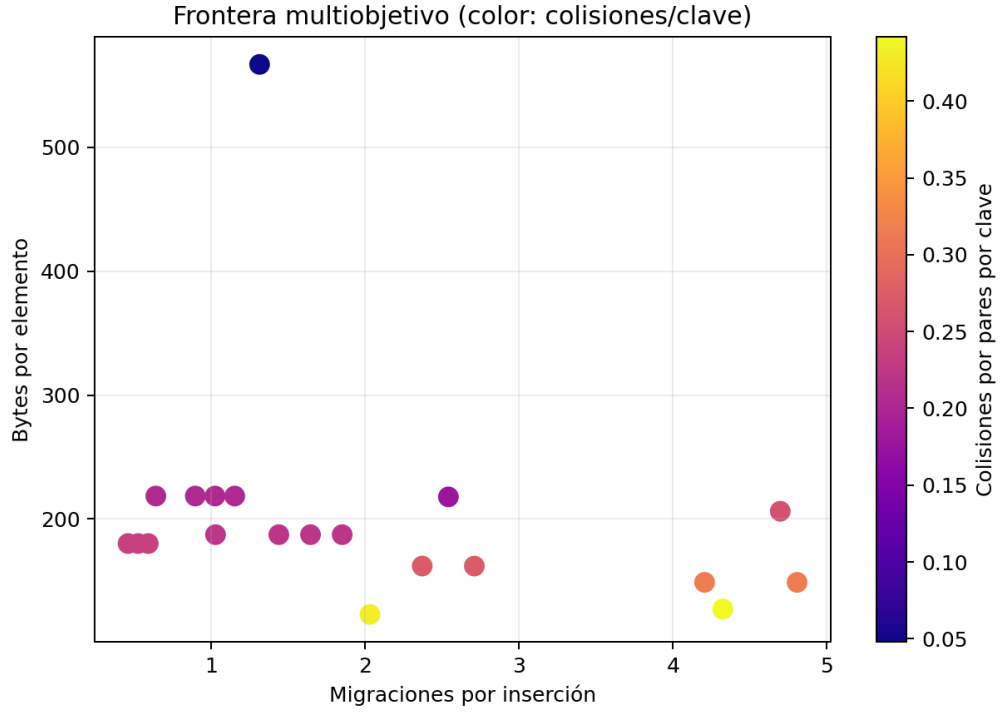


Figura 2: Compromiso entre migraciones, memoria y colisiones.

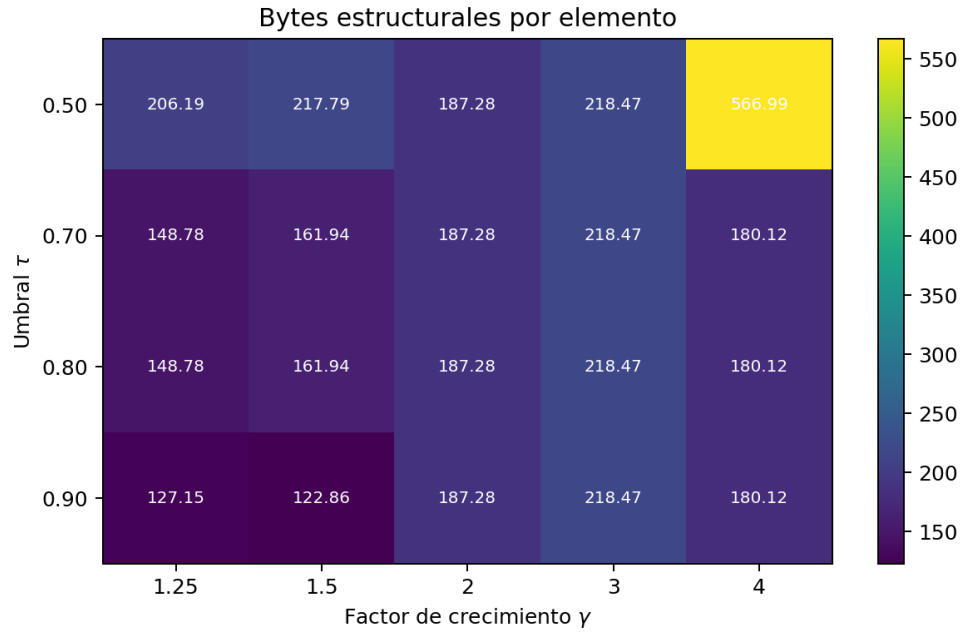


Figura 3: Memoria estructural final por elemento. Los escalones se explican por la capacidad prima alcanzada para un N fijo.

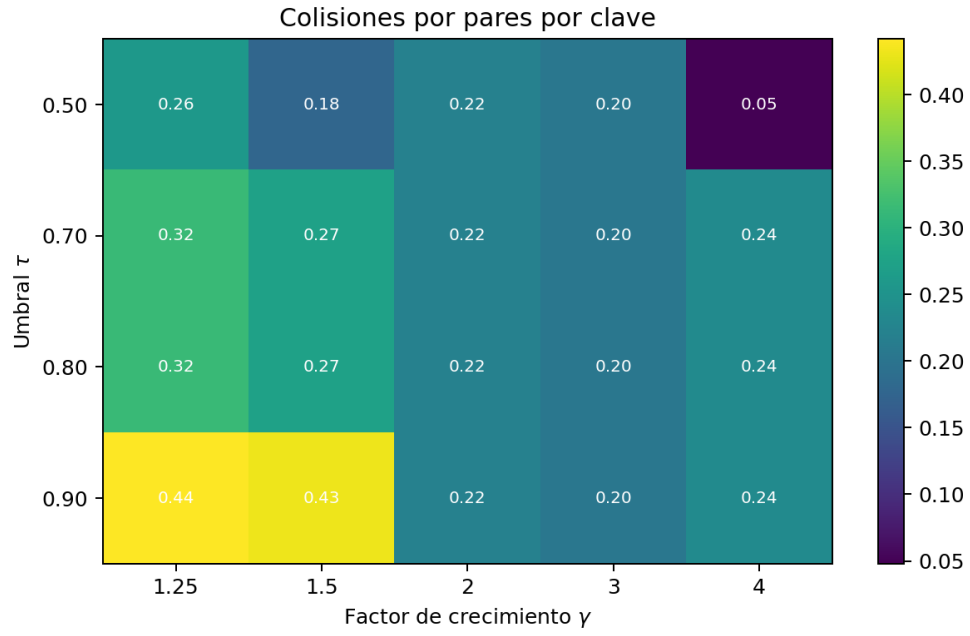


Figura 4: Colisiones por pares normalizadas. El efecto de la política opera a través de la carga final y de su trayectoria.

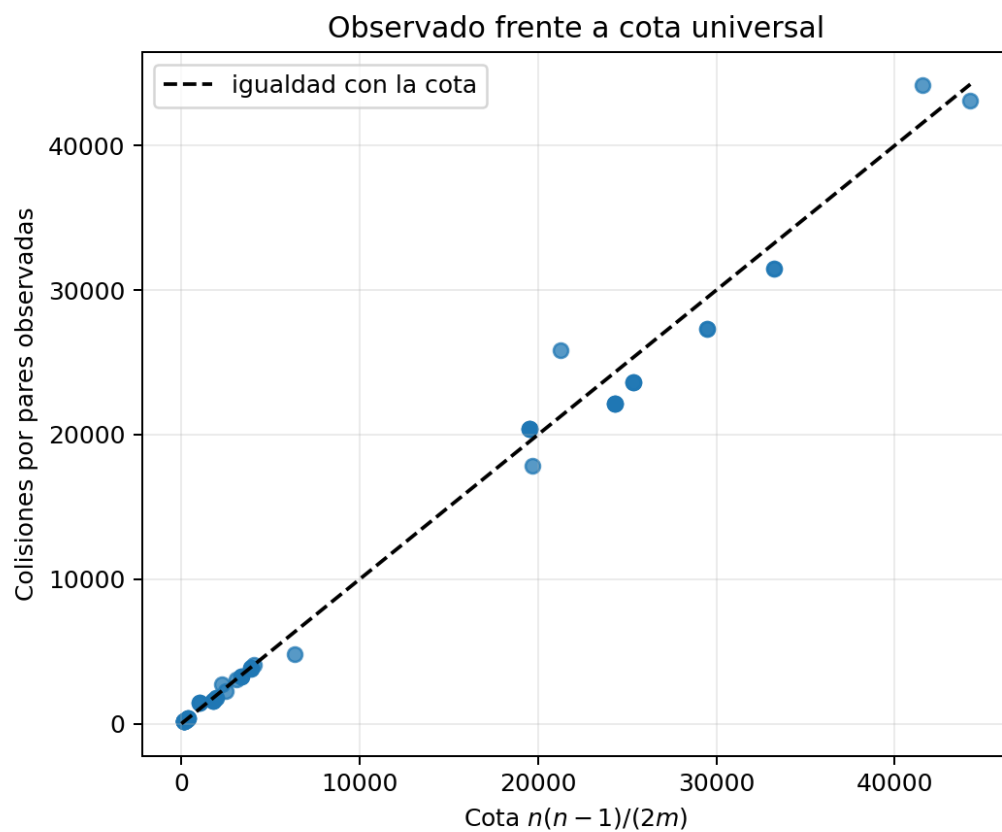


Figura 5: Pares observados frente a la cota esperada para la familia universal.

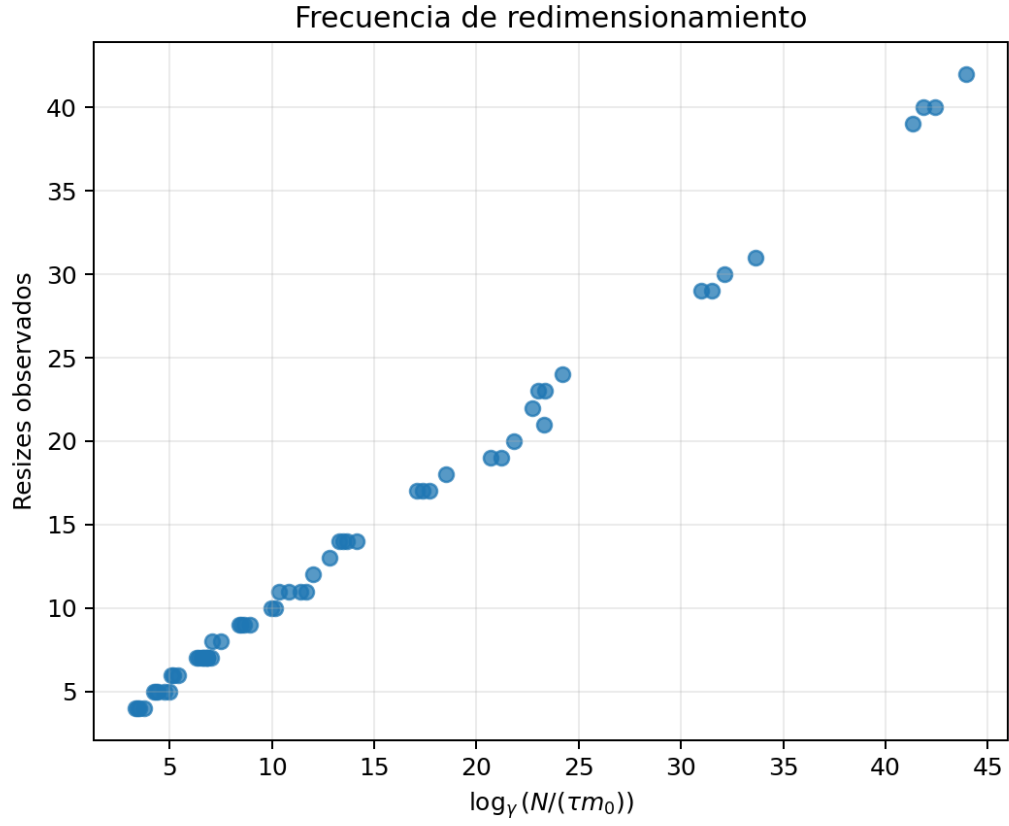


Figura 6: Número observado de resizes frente a la aproximación logarítmica.

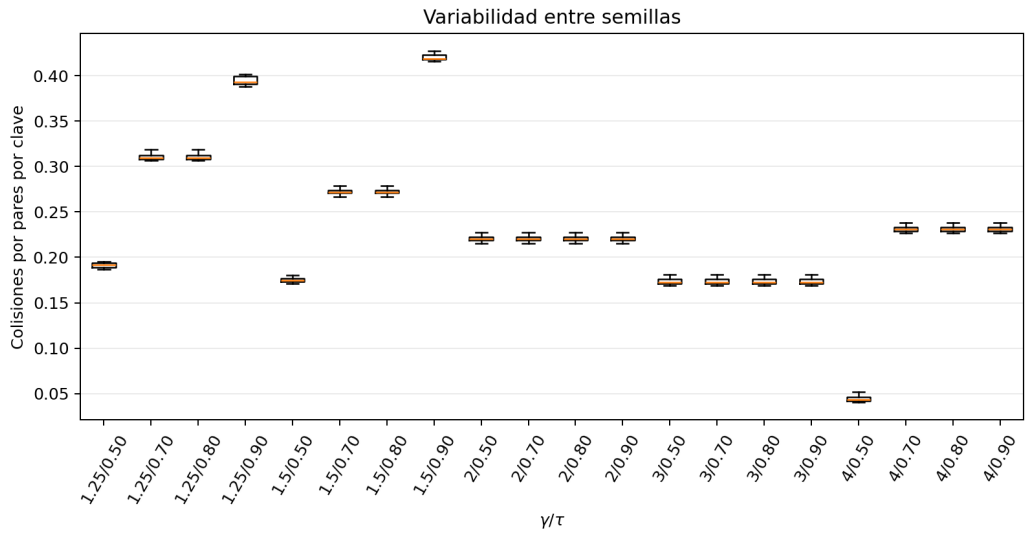


Figura 7: Variabilidad de colisiones entre las 30 semillas para $N = 100000$.

6. Discusión

La hipótesis se evalúa por patrones conjuntos, no por una sola configuración ganadora. Una caída de migraciones con γ apoya la predicción amortizada; un aumento de bytes y slots pico delimita su costo espacial. El efecto de τ debe interpretarse a través de la carga realizada: dos políticas distintas pueden terminar con capacidades diferentes y, por tanto, invertir temporalmente un orden esperado en un N concreto.

El contraste secundario ejecutó 540 corridas de sondeo lineal. Para $N = 10000$, promediando factores de crecimiento y 30 semillas, los probes por inserción fueron 1.75, 3.46 y 8.26 para $\tau = 0.50$, 0.75 y 0.90, respectivamente. El aumento pronunciado con la carga concuerda cualitativamente con el análisis clásico del probing [7]; no se compara numéricamente con C_{pares} porque es otro constructo.

La cota universal es una esperanza respecto de h , no un máximo determinista. Por ello son informativos tanto el promedio como la dispersión entre semillas. El sondeo lineal se conserva como contraste secundario: allí una “colisión” es un probe sobre un slot ocupado y aparece clustering, mientras en encadenamiento la medida principal cuenta pares de claves por bucket.

Los resultados también muestran el carácter discreto de la política. Para algunos valores consecutivos de τ , una combinación puede terminar en la misma capacidad prima y producir igual distribución final, aunque haya alcanzado los resizes en momentos distintos. Por ello no interpretamos τ como si fuera la carga final: es un umbral de decisión, mientras α es el estado efectivamente observado. La tendencia agregada de pares con τ es fuerte, pero una celda aislada puede exhibir empates o inversiones por redondeo y trayectoria.

El tiempo por operación siguió en términos generales el volumen de migraciones, pero no constituye una medición “pura” de la tabla: cada inserción fue cronometrada y `tracemalloc` permaneció activo. Esta elección mejora la observabilidad de percentiles y memoria a cambio de overhead. En consecuencia, la comparación interna entre tratamientos bloqueados es más defendible que extrapolar los nanosegundos a otra máquina o implementación.

7. Amenazas a la validez

La validez de constructo depende de no confundir eventos, pares, comparaciones y probes. La validez interna puede afectarse por GC, calentamiento, procesos del sistema y orden temporal; se mitiga con bloqueo, aleatorización y contadores independientes del hardware. La memoria de objetos Python no representa una implementación compacta en C. La validez externa está limitada a enteros, ejecución monohilo, una familia modular y distribuciones incluidas. Treinta semillas permiten describir variabilidad, pero no convierten una muestra en demostración. Finalmente, la cota $1/m$ debe comprobarse para la familia y universo usados; un diagnóstico empírico no reemplaza la prueba de Carter–Wegman.

8. Conclusiones y trabajo futuro

El estudio ofrece un procedimiento reproducible para separar los efectos directos e indirectos de γ y τ . La conclusión final sobre H1 se limita a las tendencias e intervalos observados en los CSV; no se generaliza desde Python a todas las arquitecturas. Como trabajo futuro proponemos borrados con histéresis, contracción, otros esquemas de hashing y mediciones en implementaciones compactas.

9. Declaración de uso de IA

Se utilizó ChatGPT/Codex el 8 de septiembre de 2026 para auditar ramas, estructurar el protocolo, asistir en código, pruebas, análisis y borradores LaTeX. Los documentos generados identifican explícitamente los elementos que requieren revisión humana. Las bitácoras contienen el detalle por artefacto y prompt representativo. Las fuentes científicas, resultados, decisiones y responsabilidad final corresponden al equipo; una sugerencia de IA no se considera fuente ni evidencia de autoría estudiantil.

Referencias

- [1] Michael A. Bender, Alex Conway, Martín Farach-Colton, William Kuszmaul, and Guido Tagliavini. Iceberg hashing: Optimizing many hash-table criteria at once. *Journal of the ACM*, 70(6):1–51, 2023.
- [2] Michael A. Bender, Bradley C. Kuszmaul, and William Kuszmaul. Linear probing revisited: Tombstones mark the demise of primary clustering. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science*, pages 1171–1182. IEEE, 2022.
- [3] Maximilian Böther, Lawrence Benson, Ana Klimovic, and Tilmann Rabl. Analyzing vectorized hash tables across CPU architectures. *Proceedings of the VLDB Endowment*, 16(11):2755–2768, 2023.
- [4] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Algorithms and Data Structures, WADS 1999*, volume 1663 of *Lecture Notes in Computer Science*, pages 37–48. Springer, 1999.
- [5] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.
- [6] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohmert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994.
- [7] Philippe Flajolet, Patricio Poblete, and Alfredo Viola. On the analysis of linear probing hashing. *Algorithmica*, 22:490–515, 1998.
- [8] Per-Åke Larson. Dynamic hash tables. *Communications of the ACM*, 31(4):446–457, 1988.
- [9] Yuchen Li, Qiwei Zhu, Zheng Lyu, Zhongdong Huang, and Jianling Sun. DyCuckoo: Dynamic hash tables on GPUs. In *2021 IEEE 37th International Conference on Data Engineering*, pages 744–755. IEEE, 2021.
- [10] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.
- [11] Mihai Pătraşcu and Mikkel Thorup. The power of simple tabulation hashing. *Journal of the ACM*, 59(3):14:1–14:50, 2012.
- [12] Stefan Richter, Victor Alvarez, and Jens Dittrich. A seven-dimensional analysis of hashing methods and its implications on query processing. *Proceedings of the VLDB Endowment*, 9(3):96–107, 2015.

- [13] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays. *SIAM Journal on Computing*, 53(5):1354–1380, 2024.
- [14] Robert Endre Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985.

A. Matriz Maestra de Cobertura Temática

Unidad	Subtema	Estado	Justificación técnica y evidencia
Análisis amortizado	Secuencias de operaciones	A	El costo se evalúa sobre secuencias completas de inserciones; Sec. 2.2 y CSV.
	Método agregado	A	Se suman migraciones de todos los resizes y se normalizan por N ; Sec. 2.2.
	Método contable	P	Los créditos por inserción ofrecen una interpretación de cómo financiar migraciones; no es la prueba principal.
	Método potencial	P	Se discute como alternativa; ocupación sola no captura redondeo primo e interacción (γ, τ) .
	Arreglos dinámicos	A	La reserva geométrica y frontera tiempo-espacio fundamentan el redimensionamiento; Sec. 2.3.
	Contadores y operaciones	A	Resizes ocasionales, elementos movidos y costo por operación están instrumentados en <code>src/</code> .
Probabilidad y esperanza	Probabilidad básica	A	La elección de (a, b) define el espacio aleatorio y los eventos de colisión.
	Esperanza matemática	A	Se contrasta $\mathbb{E}[C_{\text{pares}}] \leq n(n-1)/(2m)$; Sec. 2.1 y Fig. 5.
	Variables indicadoras	A	Cada par aporta una indicadora; su suma produce C_{pares} y permite linealidad de esperanza.
	Amortizado vs. esperado	A	Se distinguen explícitamente garantía de secuencia, esperanza sobre h y media empírica.
Algoritmos aleatorizados	Las Vegas	P	La tabla siempre conserva corrección; la función aleatoria afecta costo, no respuesta.
	Monte Carlo	N	No se acepta probabilidad de respuesta incorrecta: todas las inserciones se verifican.
	Probabilidad de error	N	No existe salida aproximada o errónea que deba amplificarse mediante repetición.
	Garantías de ejecución	P	Se comparan corrección determinista, costo amortizado y colisiones esperadas.
	Selección aleatoria	P	Se eligen aleatoriamente (a, b) y el orden de tratamientos, no pivotes algorítmicos.
	Hashing universal	A	Es el mecanismo aleatorizado central; <code>src/universal_hash.py</code> .

Unidad	Subtema	Estado	Justificación técnica y evidencia
P, NP y verificadores	Problemas de decisión	N	La pregunta estima rendimiento multiobjetivo, no decide un lenguaje sí/no.
	Certificados	N	Las observaciones no requieren testigo para una instancia “sí”.
	Verificadores	N	No se formula certificado; la corrección se comprueba mediante invariantes polinomiales.
	Clase P	P	Inserción y validaciones tienen algoritmos polinomiales; no se estudia completitud.
	Clase NP	P	Se explica que pertenencia exige problema de decisión y verificador; esa forma no es central aquí.
	Decisión vs. optimización	P	Elegir una política Pareto es optimización multiobjetivo, distinta de una decisión formal.
Reducciones y NP-completitud	Reducciones polinomiales	N	No se compara dificultad de lenguajes mediante transformaciones de instancias.
	Dirección de una reducción	N	No se afirma dureza; invertir una reducción no demostraría la implicación buscada.
	Prueba de NP-completitud	N	Faltan deliberadamente lenguaje, pertenencia a NP y reducción desde un problema base.
	SAT / 3-SAT	N	No modela la selección de parámetros ni la operación de la tabla.
	Clique	N	Las colisiones forman grupos por bucket, pero no equivalen a clique de una instancia arbitraria.
	Conjunto independiente	N	Evitar colisiones no se formula como selección de vértices sin adyacencias.
	Cobertura de vértices	N	No existe grafo de entrada ni objetivo de cubrir aristas.
	Ciclo hamiltoniano	N	La secuencia de buckets no exige visitar vértices una vez.
	Errores frecuentes	P	Se reconoce que una analogía estructural no constituye reducción ni equivalencia.
	Consecuencias prácticas	P	NP-completitud no implica que problemas polinomiales como esta simulación sean intratables.
Aproximación	Algoritmos de aproximación	N	El factorial mide políticas; no aproxima una solución óptima NP-difícil.
	Razón de aproximación	N	La frontera de Pareto no ofrece razón demostrable respecto de un óptimo único.
	Análisis de garantías	P	Se analizan cotas amortizadas/esperadas, que no deben confundirse con garantía de aproximación.

B. Matriz bibliográfica

Fuente	Problema/objetivo	Método	Resultado relevante	Uso y limitación
Carter–Wegman (1979)	Evitar supuestos benignos sobre claves	Familias universales y análisis probabilístico	Control de colisiones por pares	Fundamenta C_{pares} ; no garantiza probes de sondeo lineal.
Tarjan (1985)	Costear secuencias con eventos caros	Análisis amortizado	Costo medio acotado sobre secuencias	Marco para migraciones; no prescribe (γ, τ) .
Larson (1988)	Hashing dinámico en memoria	Diseño, análisis y experimentación	La expansión controla carga y reorganización	Paper núcleo más cercano; plataforma antigua.
Dietzfelbinger et al. (1994)	Hashing perfecto dinámico	Estructura aleatorizada multinivel	Consultas de peor caso y actualización esperada	Contraste teórico; estructura distinta.
Flajolet et al. (1998)	Costo del sondeo lineal	Combinatoria analítica	La carga cambia el régimen de probes	Respalda experimento secundario; no encadenamiento.
Brodnik et al. (1999)	Arreglos redimensionables eficientes	Diseño y análisis tiempo–espacio	Bajo espacio extra con acceso eficiente	Contexto de reserva; no tabla hash.
Pagh–Rodler (2004)	Búsqueda constante con dos posiciones	Cuckoo hashing	Rehash aleatorio ante ciclos	Relaciona aleatoriedad y reconstrucción; otro esquema.
Pătraşcu–Thorup (2012)	Independencia práctica del hash	Análisis de tabulación simple	Garantías dependen de la aplicación	Evita generalizar universalidad a todo probing.
Richter et al. (2015)	Comparar hashing multidimensionalmente	Benchmark experimental	No hay ganador universal	Motiva factorial y múltiples métricas.
Li et al. (2021)	Hash dinámico en GPU	Diseño concurrente y evaluación	Crecimiento depende de arquitectura	Fuente reciente; no extrapolable a Python.
Bender et al. (2022)	Sondeo lineal a alta carga	Análisis con tombstones	Reconstrucción y borrado cambian clustering	Contexto para probing; workload diferente.
Bender et al. (2023)	Optimizar varios criterios	Diseño y análisis de Iceberg hashing	Trade-offs simultáneos	Motiva Pareto; estructura más compleja.
Böther et al. (2023)	Hash vectorizado entre CPU	Benchmark multiarquitectura	Hardware altera rankings	Amenaza externa; no prueba nuestra política.
Tarjan–Zwick (2024)	Arreglos redimensionables óptimos	Cotas y construcción	Frontera tiempo–espacio refinada	Paper núcleo reciente; no define política hash.

C. Registro de búsqueda y verificación

Portal	Fecha	Consulta	Filtros	Resultado/estado
ACM DL	08-09-2026	“universal classes hash functions”, “dynamic hash tables”	Fuente primaria, venue revisado	Carter–Wegman y Larson identificados; cada integrante debe confirmar lectura.
SIAM	08-09-2026	“amortized computational complexity”, “optimal resizable arrays”	DOI/editor	Tarjan 1985 y Tarjan–Zwick 2024 identificados.
IEEE Xplore	08-09-2026	“dynamic hash tables GPU”, “linear probing tombstones”	2021–2026	Li et al. y Bender et al. seleccionados como contexto reciente.
PVLDB	08-09-2026	“analysis hashing methods”, “vectorized hash tables CPU”	Papers experimentales	Richter et al. y Böther et al. para metodología y amenazas.
SpringerLink	08-09-2026	“linear probing analysis”, “resizable arrays time space”	DOI estable	Flajolet et al. y Brod-nik et al. incorporados.

Nota de integridad. Este registro fue estructurado con asistencia de IA a partir de la investigación suministrada. Antes de la entrega, los estudiantes deben abrir los DOI, consultar los trabajos y registrar en sus bitácoras qué secciones verificaron. Una referencia sugerida por IA no cuenta por sí sola como fuente consultada.